LeetCode 1231: Divide Chocolate

1. Problem Title & Link

Title: LeetCode 1231: Divide Chocolate

Link: https://leetcode.com/problems/divide-chocolate/

2. Problem Statement (Short Summary)

You are given:

- An array sweetness
- An integer k

You must divide the chocolate into:

k + 1 pieces

using exactly k cuts.

You will eat the piece with the **minimum total sweetness**.

Your goal is to maximize the sweetness of the piece you get.

Return the maximum possible minimum sweetness.

3. Examples (Input → Output)

Example 1

Input:

sweetness = [1,2,3,4,5,6,7,8,9]

k = 5

Output:

6

Explanation:

Possible division:

[1,2,3] [4,5] [6] [7] [8] [9]

Sweetness:

6, 9, 6, 7, 8, 9

Minimum = 6

Example 2

Input:

sweetness = [5,6,7,8,9,1,2,3,4]

k = 8

Output:

1

Example 3

Input:

sweetness = [1,2,2,1,2,2,1,2,2]

k = 2

Output:

5

4. Constraints

- $1 \le$ sweetness.length $\le 10^4$
- $0 \le k < n$
- $1 \le$ sweetness[i] $\le 10^5$

Need better than:

$O(n^2)$

5. Core Concept (Pattern / Topic)

Binary Search on Answer

Related Concepts:

- Greedy Validation
- Search Space Reduction
- Partition Problems

6. Thought Process (Step-by-Step Explanation)

Brute Force

Try all possible ways to place:

k cuts

Number of combinations is enormous.

Complexity:

Exponential

Not feasible.

Key Observation

Question asks:

Maximum possible minimum sweetness

This is a classic clue for:

Binary Search on Answer

Binary Search Idea

Suppose we guess:

mid = 6

Question becomes:

Can we create at least

(k+1) pieces

where each piece

has sweetness ≥ 6 ?

If YES:

Try bigger answer

If NO:

Try smaller answer

Greedy Validation

Keep adding sweetness.

Whenever:

current_sum >= target

Form a piece.

Count pieces.

If:

pieces >= k+1

Target sweetness is achievable.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Example:
sweetness = [1,2,3,4,5,6]
k = 2
Check:
target = 6
Build pieces:
1+2+3 = 6  ✓

4+5 = 9    ✓

6 = 6      ✓
Pieces formed:
3 pieces
Required:
k+1 = 3
Possible.
```

8. Pseudocode (Language Independent)

```
low = min(sweetness)

high = sum(sweetness)

while low <= high:

    mid = (low + high)//2

    if canMake(mid):

        answer = mid
        low = mid + 1

    else:

        high = mid - 1

return answer
Validation:
count pieces greedily

if pieces >= k+1:
    return True

return False
```

9. Code Implementation

✅ **Python**

```
class Solution:

    def maximizeSweetness(self, sweetness, k):

        def canMake(target):

            pieces = 0
            curr = 0

            for value in sweetness:

                curr += value

                if curr >= target:
                    pieces += 1
                    curr = 0

            return pieces >= k + 1

        low = min(sweetness)
```

```
        high = sum(sweetness)

        answer = 0

        while low <= high:

            mid = (low + high) // 2

            if canMake(mid):
                answer = mid
                low = mid + 1
            else:
                high = mid - 1

        return answer
```

✅ **Java**

```
class Solution {

    public int maximizeSweetness(int[] sweetness, int k) {

        int low = Integer.MAX_VALUE;
        int high = 0;

        for (int value : sweetness) {
            low = Math.min(low, value);
            high += value;
        }

        int answer = 0;

        while (low <= high) {

            int mid = low + (high - low) / 2;

            if (canMake(sweetness, k, mid)) {
                answer = mid;
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        return answer;
    }

    private boolean canMake(
        int[] sweetness,
```

```
        int k,
        int target
    ) {

        int pieces = 0;
        int curr = 0;

        for (int value : sweetness) {

            curr += value;

            if (curr >= target) {
                pieces++;
                curr = 0;
            }
        }

        return pieces >= k + 1;
    }
}
```

10. Time & Space Complexity

Time Complexity

Binary Search:

O(log(sum(sweetness)))

Validation:

O(n)

Overall:

O(n log(sum))

Space Complexity

O(1)

11. Common Mistakes / Edge Cases

Mistake 1

Using:

pieces == k+1

Instead of:

pieces >= k+1

More pieces are acceptable.

Mistake 2

Trying all partitions.

Will TLE.

Mistake 3

Using average sweetness.

This problem is not solved by averages.

12. Detailed Dry Run (Step-by-Step Table)

Input:

sweetness=[1,2,3,4,5,6]

k=2

Check:

mid = 6

Value	Running Sum	Piece Formed?	Pieces
1	1	No	0
2	3	No	0
3	6	Yes	1
4	4	No	1
5	9	Yes	2
6	6	Yes	3

Result:

3 >= k+1

Valid.

13. Common Use Cases (Real-Life / Interview)

Resource Allocation

Divide resources among teams while maximizing minimum allocation.

Load Balancing

Split workloads fairly.

Budget Distribution

Ensure every department gets at least a minimum budget.

Storage Partitioning

Allocate disk blocks evenly.

14. Common Traps (Important!)

- Binary searching indices instead of answer
- Using exact piece count
- Missing greedy validation
- Incorrect low/high boundaries

15. Builds To (Related LeetCode Problems)

1. LC 410 — Split Array Largest Sum
2. LC 875 — Koko Eating Bananas
3. LC 1011 — Capacity To Ship Packages
4. LC 1283 — Smallest Divisor Given Threshold
5. LC 1231 — Divide Chocolate

These are classic Binary Search on Answer problems.

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force Partitions	Exponential	O(n)	Impossible
DP	$O(n^2)$	$O(n^2)$	Heavy
Binary Search + Greedy	O(n log(sum))	O(1)	Optimal

17. Why This Solution Works (Short Intuition)

If a minimum sweetness value is achievable, then all smaller values are also achievable. This monotonic property allows binary search on the answer. Greedy partitioning efficiently checks feasibility.

18. Variations / Follow-Up Questions

Follow-Up 1

What if we need:

Minimum possible maximum piece?

Answer:

LC 410 Split Array Largest Sum

Follow-Up 2

What if cuts can be rearranged dynamically?

Use DP.